

Universidad del Valle de Guatemala

Facultad de Ingeniería

CC3067 Redes — Semestre 2, 2026

Proyecto 1 — Entrega final

Uso de un protocolo existente: Model Context Protocol

Chatbot MCP para control de abastecimiento

Integrantes:

Daniel Estrada – 20853

Guatemala, 15 de septiembre de 2026

1. Introducción

1.1 Alcance del proyecto

Este proyecto implementa un chatbot que actúa como anfitrión de Model Context Protocol (MCP) y consume herramientas publicadas por varios servidores. El anfitrión conversa con un modelo de lenguaje a través de la API Messages de Anthropic, descubre las herramientas disponibles en tiempo de ejecución y coordina su invocación según lo que el modelo solicita.

La implementación del protocolo es manual. Todo el formato e intercambio de mensajes JSON-RPC 2.0 está escrito en el repositorio: no se utiliza un SDK de MCP, ni FastMCP, ni el SDK de Anthropic. Las llamadas a la API del modelo se realizan directamente sobre HTTP.

El proyecto funciona a nivel de terminal, tal como exige el enunciado, y adicionalmente incluye una consola web de operaciones que expone la misma evidencia de protocolo en el navegador.

1.2 Funcionalidades implementadas

- Conexión con un LLM a nivel de su API, capaz de responder preguntas generales de su base de entrenamiento.
- Contexto conversacional dentro de una sesión, conservando turnos de usuario, asistente, uso de herramienta y resultado de herramienta.
- Registro visible y persistente de todas las solicitudes y respuestas MCP, en pantalla y en `logs/mcp-interactions.jsonl`.
- Uso de los servidores MCP oficiales Filesystem y Git, con un escenario reproducible sobre un repositorio descartable.
- Servidor MCP propio, basado en un caso de uso industrial, ejecutándose localmente sobre `stdio`.
- El mismo servidor desplegado de forma remota en Render, consumido por el chatbot exactamente igual que el local.
- Captura y análisis con Wireshark de la comunicación entre el anfitrión y el servidor remoto.

2. Arquitectura

2.1 Actores del protocolo

El sistema distingue los tres actores que define MCP. El anfitrión es el chatbot de terminal, que mantiene la conversación y decide cuándo invocar una herramienta. Cada cliente mantiene la conexión con un servidor y conoce cómo hablarle. Los servidores son los procesos que ejecutan las acciones: los dos oficiales, el propio servidor de abastecimiento en local y ese mismo servidor en la nube.

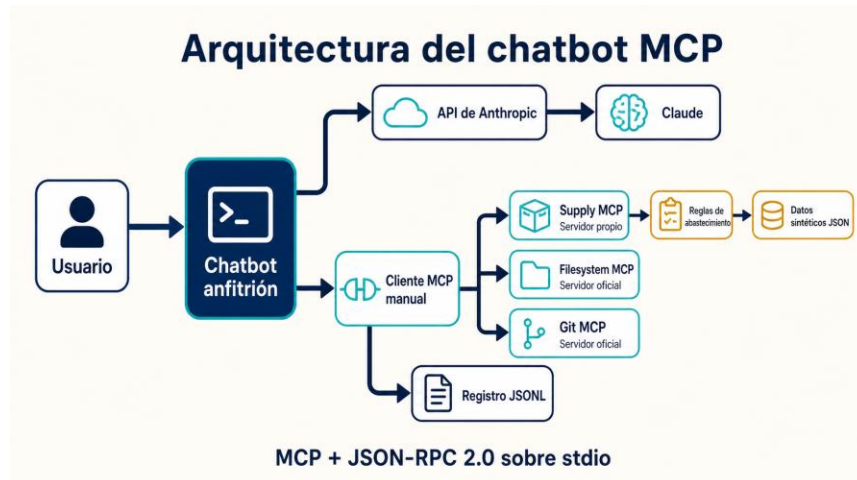


Figura 1. Arquitectura del anfitrión, sus clientes MCP y los servidores local, oficiales y remoto.

2.2 Separación entre protocolo y transporte

La decisión de diseño central fue separar el ciclo de vida de MCP del medio por el que viajan los bytes. El cliente construye, numera y correlaciona mensajes JSON-RPC sin saber si viajan por tuberías del sistema operativo o por la red. El transporte solo mueve mensajes.

Gracias a esa separación, el servidor local y el remoto comparten el mismo registro de herramientas, las mismas reglas de negocio y el mismo manejador de JSON-RPC; únicamente cambia el transporte que los envuelve. Esto es lo que permite que el chatbot use el servidor remoto sin ninguna ruta de código especial.

3. Especificación del servidor MCP desarrollado

3.1 Caso de uso industrial

El caso de uso, consultado previamente con el catedrático, es la planificación de compras de abastecimiento alimentario. Un planificador necesita saber qué materiales están por quedarse sin existencias, por qué, y cuánto debe comprar. El servidor publica esa capacidad como herramientas MCP para que el chatbot pueda responder preguntas operativas y justificar cada número.

Todos los datos son sintéticos y se cargan desde archivos JSON locales; no hay conexión con ningún sistema real. Las recomendaciones se calculan con reglas determinísticas dentro del servidor: el modelo de lenguaje interpreta y explica, pero no calcula inventario.

3.2 Herramientas publicadas

El servidor `synthetic-supply-control` versión 1.0.0 negocia la revisión MCP 2025-11-25 y publica cinco herramientas:

Herramienta	Parámetros	Resultado
<code>list_inventory_risks</code>	<code>center</code> (opcional), <code>horizon_days</code> (1–30, por omisión 7), <code>statuses</code> (opcional), <code>limit</code> (1–100, por omisión 20)	Materiales ordenados por riesgo de abastecimiento
<code>get_material_status</code>	<code>center</code> (requerido), <code>material_code</code> (requerido)	Existencias, demanda, cobertura, quiebre proyectado, recepciones programadas y parámetros maestros
<code>get_purchase_recommendations</code>	<code>center</code> , <code>only_with_order</code> , <code>only_with_residual_shortage</code> , <code>limit</code> (1–100)	Sugerencias de compra determinísticas
<code>explain_purchase_recommendation</code>	<code>center</code> (requerido), <code>material_code</code> (requerido)	Insumos, fórmula, redondeo y riesgo residual de una compra recomendada
<code>get_supply_data_status</code>	Ninguno	Fecha operativa, frescura de fuentes, conteos y advertencias de calidad de datos

Los centros válidos son DC-PROD y DC-STORES. Cada herramienta declara su esquema de entrada en `tools/list`, de modo que el modelo conoce los parámetros sin que estén codificados en el anfitrión. Los resultados incluyen `structuredContent` además del texto, lo que permite a las interfaces tabular los registros en lugar de depender de la prosa del modelo.

3.3 Contrato de la interfaz local

Propiedad	Valor
Nombre del servidor	<code>synthetic-supply-control</code>
Versión	1.0.0
Revisión MCP	2025-11-25
Transporte	Local <code>stdio</code>
Punto de entrada	<code>node dist/src/mcp/supply-server-entry.js</code>
Endpoint de red	Ninguno; mensajes delimitados por salto de línea en <code>stdin/stdout</code>
Codificación	JSON UTF-8, un objeto JSON-RPC por línea

3.4 Contrato de la interfaz remota

El mismo servidor y las mismas reglas de negocio se exponen sobre HTTPS mediante un subconjunto sin streaming del transporte Streamable HTTP, también implementado a mano:

Propiedad	Valor
Transporte	HTTPS solicitud/respuesta
Punto de entrada	node dist/src/mcp/supply-server-http-entry.js
Endpoint MCP	POST /mcp
Health check	GET /healthz
Cierre de sesión	DELETE /mcp con Mcp-Session-Id; responde 204, o 404 si no existe
Consola web	GET /
API de la consola	GET /api/bootstrap, POST /api/analyze, POST /api/chat
Autenticación	Authorization: Bearer <MCP_API_KEY>, comparada en tiempo constante
Cabecera de sesión	Mcp-Session-Id, devuelta por initialize y reenviada por el cliente
Cabecera de versión	Mcp-Protocol-Version: 2025-11-25 tras la inicialización
Codificación	application/json UTF-8 con JSON-RPC 2.0

El endpoint está expuesto a Internet, así que el servidor acota el estado que un cliente puede crear: una sesión caduca a los 30 minutos sin tráfico, se admiten como máximo 256 sesiones simultáneas —más allá responde 503 con Retry-After— y un cuerpo mayor a 256 KiB se rechaza con 413 en lugar de almacenarse en memoria. Las señales SIGTERM y SIGINT cierran el escuchador antes de terminar el proceso, de modo que un redespliegue no corta la solicitud en curso.

3.5 Despliegue

El `Dockerfile` compila TypeScript en una etapa y ejecuta solo los artefactos necesarios sobre Node.js 22. El archivo `render.yaml` describe el servicio remoto, su health check y la generación de la credencial como variable secreta. Como alternativa, `scripts/deploy-cloud-run.ps1` despliega la misma imagen en Google Cloud Run. Ambos flujos usan una sola instancia, ya que las sesiones viven en memoria.

4. Ciclo de mensajes JSON-RPC

Cada solicitud lleva `jsonrpc`, `id`, `method` y `params`. Cada respuesta conserva el mismo `id` y contiene `result` o `error`. Las notificaciones son mensajes sin `id` y por definición no reciben respuesta; el servidor contesta 202 Accepted con cuerpo vacío.

El intercambio completo para responder una pregunta operativa es el siguiente:



Figura 2. Secuencia de mensajes entre el anfitrión, el cliente MCP y el servidor de abastecimiento.

Un ejemplo real de solicitud de invocación, tomado del registro de auditoría:

```
{ "jsonrpc": "2.0", "id": 4, "method": "tools/call",
  "params": { "name": "get_material_status",
    "arguments": { "center": "DC-PROD",
      "material_code": "SYN-PROD-004" } } }
```

5. Análisis de la comunicación con Wireshark

5.1 Clasificación de los mensajes JSON-RPC

La comunicación entre el anfitrión y el servidor remoto se compone de tres clases de mensaje. La tabla siguiente indica a qué clase corresponde cada uno y qué función cumple:

Clase	Método o forma	Lleva id	Función
Sincronización	initialize	Sí	Negocia versión del protocolo, capacidades e identidad
Sincronización	notifications/initialized	No	Confirma que el cliente terminó su inicialización
Solicitud	tools/list	Sí	Descubre las cinco herramientas y sus esquemas
Respuesta	result.tools	Mismo id	Devuelve las definiciones de herramienta
Solicitud	tools/call	Sí	Invoca una herramienta con sus argumentos
Respuesta	result.content	Mismo id	Devuelve texto y contenido estructurado

Las dos primeras filas son los mensajes de sincronización: establecen la sesión antes de que cualquier herramienta pueda usarse. El servidor rechaza `tools/list` y `tools/call` si no ha recibido antes `notifications/initialized`.

5.2 Metodología de captura

El servidor remoto se sirve sobre HTTPS, de modo que una captura directa muestra únicamente registros TLS cifrados y no permite leer los cuerpos JSON-RPC. Para poder clasificar los mensajes por número de trama, la captura se realiza registrando las claves de sesión TLS y descifrando el flujo en Wireshark. El procedimiento es reproducible:

1. Ejecutar `npm run demo:remote` una vez para despertar el servicio de Render, que en el plan gratuito se suspende tras un periodo de inactividad.
2. Iniciar la captura en Wireshark sobre la interfaz que tiene la ruta hacia Internet.
3. Ejecutar `npm run capture:remote`. Es el mismo cliente contra el mismo servidor remoto, pero Node escribe los secretos de sesión TLS en `tmp/tls-keys.log`.
4. Detener la captura y guardarla como `evidence/remote-mcp.pcapng`.
5. En Wireshark, abrir Preferences > Protocols > TLS y fijar «(Pre)-Master-Secret log filename» en `tmp/tls-keys.log`.
6. Aplicar el filtro de visualización y anotar, por número de trama, qué mensaje es de sincronización, cuál es solicitud y cuál es respuesta.

El filtro utilizado para aislar el tráfico del servicio es:

```
dns.qry.name contains "supply-control-mcp.onrender.com"  
|| ip.addr == 216.24.57.7
```

El archivo de claves descifra únicamente esa captura, pero el tráfico descifrado incluye la cabecera `Authorization: Bearer`. Por eso el archivo no se publica en el repositorio y la credencial se rota después de la demostración.

5.3 Evidencia observada

La captura canónica contiene 102 paquetes y registra un intercambio completo contra el servicio autenticado en Render:

Elemento	Valor observado
URL del servicio	https://supply-control-mcp.onrender.com/mcp
Dirección del cliente	192.168.0.18 (interfaz Wi-Fi)
Dirección del servidor	216.24.57.7, puerto TCP 443
Consulta y respuesta DNS	Tramas 43–44
Saludo TCP (SYN, SYN-ACK, ACK)	Tramas 45–47
TLS ClientHello y ServerHello	Tramas 49 y 51
Datos de aplicación de la demostración	Tramas 73–101
Cierre de la conexión HTTPS	Tramas 22, 24 y 25

A nivel de aplicación, la ejecución verificada produjo `initialize` con HTTP 200 y creación de sesión MCP, `notifications/initialized` con HTTP 202, `tools/list` con HTTP 200 devolviendo cinco herramientas, y `tools/call` sobre `get_supply_data_status` con HTTP 200 sin error.

Con el archivo de claves cargado, cada uno de esos mensajes queda localizado en una trama concreta de la captura:

Mensaje	Clase	Trama
<code>initialize</code>	Sincronización	—
Respuesta a <code>initialize</code>	Respuesta	—
<code>notifications/initialized</code>	Sincronización	—
<code>tools/list</code>	Solicitud	—
Respuesta a <code>tools/list</code>	Respuesta	—
<code>tools/call</code>	Solicitud	—
Respuesta a <code>tools/call</code>	Respuesta	—

PENDIENTE: completar la columna «Trama» ejecutando `npm run capture:remote`, cargando `tmp/tls-keys.log` en Wireshark y leyendo los números de trama de los mensajes descifrados. Sustituir también la Figura 3 por una captura de pantalla con los cuerpos JSON-RPC visibles. Eliminar esta nota antes de entregar.

Evidencia Wireshark - MCP remoto en Render			
Captura real: evidence/remote-mcp.pcapng Wi-Fi 14-09-2026			
Cliente 192.168.0.18	Servidor 216.24.57.7:443	Paquetes 102	Transporte TCP + TLS 1.3
Trama	Protocolo	Dirección	Interpretación
43	DNS	192.168.0.18 -> DNS	AAAA supply-control-mcp.onrender.com
44	DNS	DNS -> 192.168.0.18	Respuesta con CNAME de Render/Cloudflare
45	TCP	192.168.0.18 -> 216.24.57.7:443	SYN
46	TCP	216.24.57.7:443 -> 192.168.0.18	SYN ACK
47	TCP	192.168.0.18 -> 216.24.57.7:443	ACK
49	TLS 1.2	Cliente -> Render	Client Hello; SNI visible
51	TLS 1.3	Render -> cliente	Server Hello + Change Cipher Spec
55-58	TLS 1.3	Cliente <-> Render	Handshake final y datos cifrados
73-101	TLS 1.3	Cliente <-> Render	Solicitudes y respuestas MCP cifradas
22,24,25	TCP	Cliente <-> Render	Cierre FIN/ACK de conexión HTTPS

El contenido JSON-RPC y el Bearer token no son visibles en Wireshark porque HTTPS los cifra.

Figura 3. Correlación de la captura: resolución DNS, establecimiento TCP, negociación TLS e intercambio de aplicación con el servidor remoto.

5.4 Capa de enlace

En la red local, la trama transporta el paquete IP hacia el siguiente salto. Sobre Ethernet y Wi-Fi contiene las direcciones MAC de origen y destino y un EtherType que identifica IPv4 o IPv6. Es importante notar que la MAC de destino observada corresponde al gateway local y no al servidor de Render: la dirección física solo tiene alcance dentro del enlace, y se reescribe en cada salto del camino. La captura se realizó sobre la interfaz Wi-Fi activa, y Npcap entregó a Wireshark las tramas observables de esa interfaz.

5.5 Capa de red

IP proporciona el direccionamiento y el encaminamiento entre el cliente y el frontend de Render. Cada paquete lleva las direcciones IP de origen y destino, un límite de saltos y el identificador del protocolo de transporte. Antes de cualquier conexión, DNS resuelve el nombre del servicio a una dirección alcanzable, lo que en la captura aparece como el primer intercambio útil. La dirección observada pertenece a infraestructura compartida de Render y puede cambiar entre ejecuciones, por lo que el filtro se ancla también al nombre consultado por DNS.

5.6 Capa de transporte

TCP establece una conexión confiable mediante el saludo de tres vías. Los números de secuencia y las confirmaciones ordenan los bytes y permiten detectar pérdidas; la ventana regula el flujo. El puerto destino es 443 y el puerto origen es efímero. Sobre TCP se ejecuta TLS, que no es una capa del modelo pero sí determina lo que resulta visible: a partir del intercambio de claves, todo el contenido de aplicación viaja cifrado, y el análisis de la capa superior solo es posible con las claves de sesión registradas.

5.7 Capa de aplicación

HTTP transporta cada mensaje MCP como un `POST /mcp` con cuerpo `application/json`. Dentro de ese cuerpo viaja un único objeto JSON-RPC 2.0. HTTPS protege tanto las cabeceras sensibles, incluida la de autorización, como el contenido JSON-RPC. La correlación entre solicitud y respuesta no depende del orden de llegada de los paquetes sino del campo `id`, que el servidor devuelve sin modificar; esta es justamente la función que JSON-RPC añade sobre el transporte. En paralelo, el registro de auditoría conserva las solicitudes, notificaciones y respuestas completas con marca de tiempo, lo que permite contrastar lo capturado en la red con lo que el anfitrión creyó enviar y recibir.

6. Dificultades y soluciones

- Reutilizar la misma lógica en local y en remoto. Se resolvió separando el protocolo del transporte, de modo que el ciclo MCP es idéntico sobre `stdio` y sobre HTTP.
- Colisión de nombres entre servidores. Los servidores oficiales publican herramientas distintas, así que se antepone el nombre del servidor a cada herramienta descubierta.
- El servidor Git oficial no implementa `git_init`. El escenario prepara un repositorio descartable y ejecuta por MCP las operaciones que sí evalúa el enunciado.
- El transporte remoto requiere estado de sesión, que `stdio` no necesita. Se añadió la cabecera `Mcp-Session-Id` y se acotó el despliegue a una instancia, con caducidad y límite de sesiones para no acumular estado indefinidamente.
- Los errores HTTP dejaban al cliente esperando. Ahora un código de error falla de inmediato en lugar de agotar el tiempo de espera.
- El plan gratuito de Render suspende el servicio y el primer contacto puede tardar cerca de un minuto. La demostración remota ahora sondea `/healthz` hasta que el servicio responde antes de medir el protocolo.
- HTTPS impedía clasificar los mensajes en la captura. Se resolvió registrando las claves de sesión TLS con `--tls-keylog` y descifrando en Wireshark, en lugar de renunciar al análisis o afirmar contenidos no observados.

7. Conclusiones y comentarios

MCP cumple su propósito: permite que un anfitrión coordine herramientas heterogéneas sin acoplar la lógica del modelo a cada servidor. El mismo chatbot usa los servidores oficiales de Filesystem y Git y un servidor propio sin contener una sola línea específica para ninguno de ellos, porque todo lo que necesita saber lo obtiene en tiempo de ejecución mediante `initialize` y `tools/list`.

La división de responsabilidades entre JSON-RPC y el transporte resultó ser la lección de diseño más valiosa. JSON-RPC resuelve la correlación entre solicitudes y respuestas mediante el campo

id; el transporte solo resuelve el movimiento de bytes. Esa frontera es lo que hizo posible pasar de un proceso local a un servicio HTTPS en la nube sin modificar el ciclo del protocolo, y explica por qué el mismo servidor funciona en ambos modos.

Implementar el protocolo manualmente, sin SDK, obligó a leer la especificación con detalle y expuso aspectos que una biblioteca habría ocultado: la diferencia entre solicitud y notificación, la obligación de no responder a una notificación, la negociación de versión y la necesidad de estado de sesión en cuanto el transporte deja de ser un proceso hijo.

Desde la perspectiva de redes, el ejercicio dejó claro que la observabilidad de una aplicación distribuida depende de la capa en la que se mire. Un analizador de paquetes muestra con precisión el enlace, la red y el transporte, pero frente a TLS la capa de aplicación es opaca por diseño. La conclusión honesta no es que el análisis sea imposible, sino que requiere una decisión explícita: registrar las claves de sesión y descifrar, o correlacionar con los registros de la aplicación. Afirmar que un paquete cifrado contiene un método concreto, sin ninguna de esas dos cosas, sería una inferencia y no una observación.

Finalmente, exponer un servicio a Internet cambió las prioridades del código. Un servidor que en local solo debía ser correcto pasó a necesitar autenticación comparada en tiempo constante, caducidad de sesiones, límites de tamaño de solicitud y apagado ordenado. Ninguna de esas preocupaciones existe cuando el servidor es un proceso hijo en la misma máquina.

8. Referencias

JSON-RPC Working Group. (2010). JSON-RPC 2.0 Specification.

<https://www.jsonrpc.org/specification>

Anthropic. (2025). Model Context Protocol — Architecture.

<https://modelcontextprotocol.io/docs/learn/architecture>

Anthropic. (2025). Model Context Protocol Specification, revisión 2025-11-25.

<https://modelcontextprotocol.io/specification/2025-11-25>

Model Context Protocol. (2025). Reference servers. GitHub.

<https://github.com/modelcontextprotocol/servers>

Anthropic. (2025). Messages API. <https://docs.anthropic.com/en/api/messages>

Wireshark Foundation. (2026). TLS decryption using a pre-master secret log.

<https://wiki.wireshark.org/TLS>